

AI ASS 13.4

Name: B.Sanjana

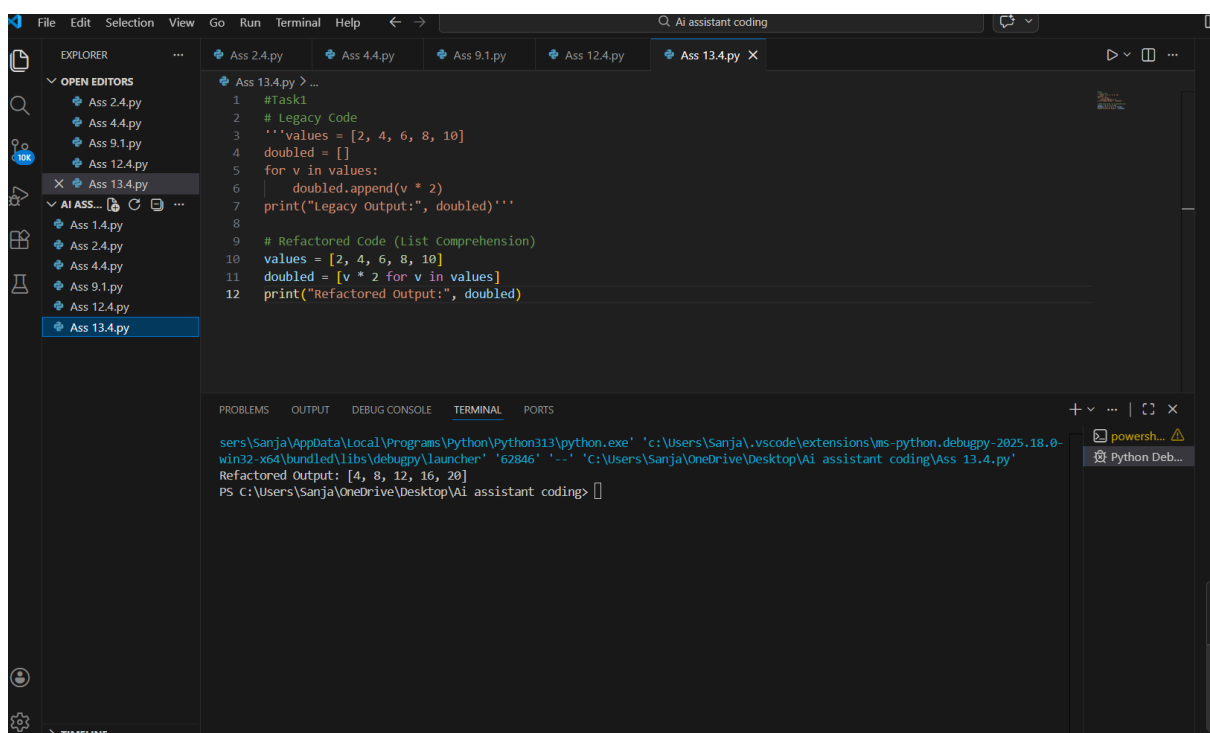
H.T.NO: 2303A52306

Batch: 43

Task1:

Prompt:

Refactor the Python code that uses a loop to double the values in a list. Suggest a more Pythonic approach using list comprehension while keeping the output the same.



The screenshot shows the Visual Studio Code editor interface. The Explorer panel on the left shows a project named 'AI assistant coding' with several Python files. The file 'Ass 13.4.py' is open in the editor. The code in the editor is as follows:

```
1 #Task1
2 # Legacy Code
3 '''values = [2, 4, 6, 8, 10]
4 doubled = []
5 for v in values:
6     doubled.append(v * 2)
7 print("Legacy Output:", doubled)'''
8
9 # Refactored Code (List Comprehension)
10 values = [2, 4, 6, 8, 10]
11 doubled = [v * 2 for v in values]
12 print("Refactored Output:", doubled)
```

The bottom panel shows the TERMINAL output:

```
Users\Sanja\AppData\Local\Programs\Python\python313\python.exe 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '62846' '-.' 'C:\Users\Sanja\OneDrive\Desktop\AI assistant coding\Ass 13.4.py'
Refactored Output: [4, 8, 12, 16, 20]
PS C:\Users\Sanja\OneDrive\Desktop\AI assistant coding>
```

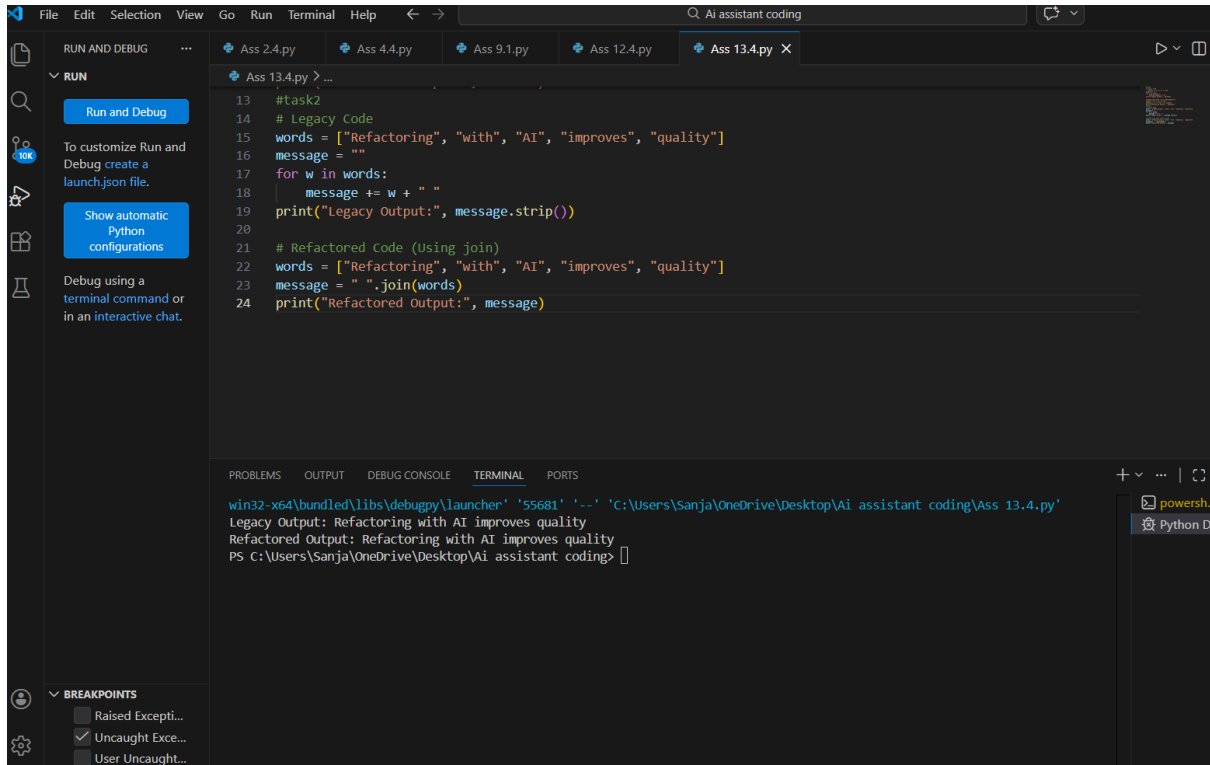
Observation:

The legacy code uses a loop to double the values in a list. After refactoring using list comprehension, the code becomes shorter, more readable, and efficient while producing the same output.

Task2

Prompt:

Improve the readability and efficiency of the Python code that builds a sentence using repeated string concatenation. Suggest a better Python method while keeping the output unchanged.



The screenshot shows a Python IDE with a dark theme. The editor displays a Python script with two versions of code. The first version, labeled '# Legacy Code', uses string concatenation to build a sentence. The second version, labeled '# Refactored Code (Using join)', uses the `join()` method. The terminal window at the bottom shows the output of both versions, which is identical: 'Legacy Output: Refactoring with AI improves quality' and 'Refactored Output: Refactoring with AI improves quality'.

```
13 #task2
14 # Legacy Code
15 words = ["Refactoring", "with", "AI", "improves", "quality"]
16 message = ""
17 for w in words:
18     message += w + " "
19 print("Legacy Output:", message.strip())
20
21 # Refactored Code (Using join)
22 words = ["Refactoring", "with", "AI", "improves", "quality"]
23 message = " ".join(words)
24 print("Refactored Output:", message)
```

win32-x64\bundled\libs\debugpy\launcher '55681' '-' 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 13.4.py'
Legacy Output: Refactoring with AI improves quality
Refactored Output: Refactoring with AI improves quality
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>

Observation:

The legacy code builds a sentence using repeated string concatenation, which is inefficient. Using the `join()` method improves readability and performance while maintaining the same result.

Task3

Prompt:

Refactor the Python code that manually checks dictionary keys. Suggest a safer way to access dictionary values using built-in Python methods while maintaining the same behavior for missing keys.

```
25 #Task3
26 # Legacy Code
27 config = {"host": "localhost", "port": 8080}
28 if "timeout" in config:
29     print("Legacy Output:", config["timeout"])
30 else:
31     print("Legacy Output: Default timeout used")
32 # Refactored Code (Using get)
33 config = {"host": "localhost", "port": 8080}
34 timeout = config.get("timeout", "Default timeout used")
35 print("Refactored Output:", timeout)
```

```
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c:; cd 'c:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-servers\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-servers\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '51717' '--' 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 13.4.py'
Legacy Output: Default timeout used
Refactored Output: Default timeout used
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding>
```

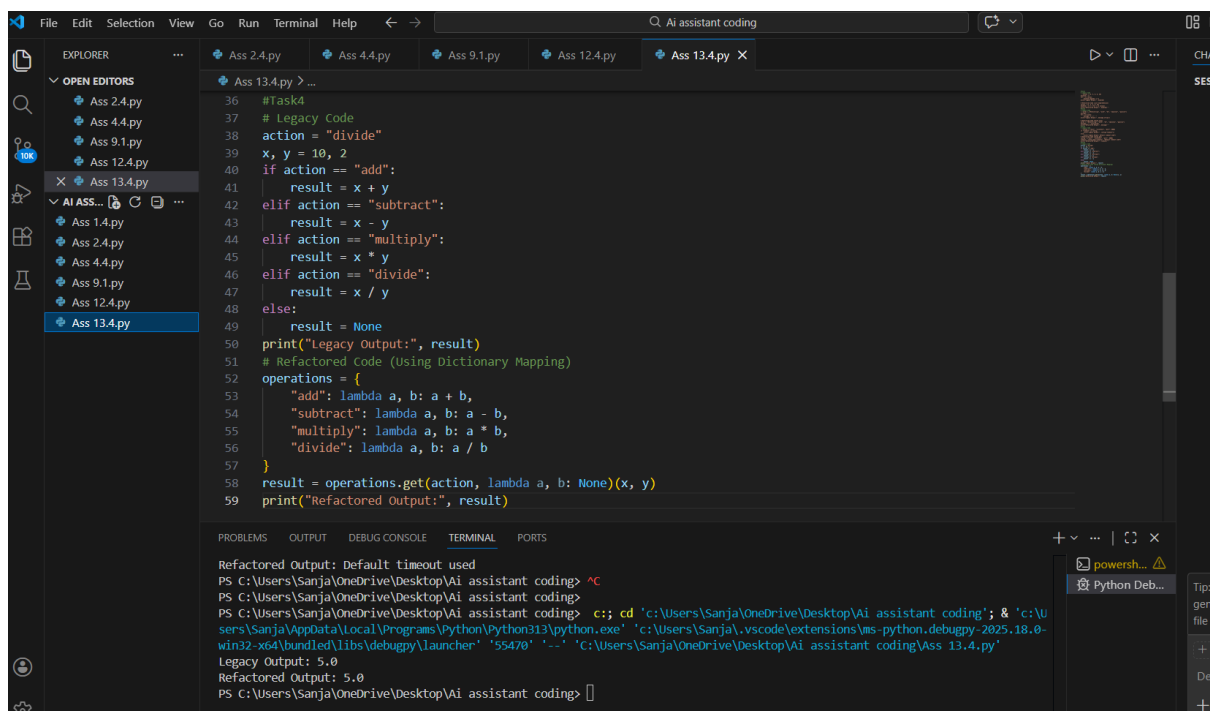
Observation:

The original code manually checks if a key exists in the dictionary. Using the `get()` method provides a safer and more concise way to access dictionary values without causing errors.

Task4

Prompt:

Refactor the Python code that uses multiple if–elif conditions to perform operations. Suggest a cleaner and scalable approach using mapping techniques such as dictionaries while preserving the functionality.



```
36 #Task4
37 # Legacy Code
38 action = "divide"
39 x, y = 10, 2
40 if action == "add":
41     result = x + y
42 elif action == "subtract":
43     result = x - y
44 elif action == "multiply":
45     result = x * y
46 elif action == "divide":
47     result = x / y
48 else:
49     result = None
50 print("Legacy Output:", result)
51 # Refactored code (Using Dictionary Mapping)
52 operations = {
53     "add": lambda a, b: a + b,
54     "subtract": lambda a, b: a - b,
55     "multiply": lambda a, b: a * b,
56     "divide": lambda a, b: a / b
57 }
58 result = operations.get(action, lambda a, b: None)(x, y)
59 print("Refactored Output:", result)
```

Refactored Output: Default timeout used
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> ^C
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> c:; cd 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding'; & 'c:\Users\Sanja\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\Sanja\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '55470' '-.' 'C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding\Ass 13.4.py'
Legacy Output: 5.0
Refactored Output: 5.0
PS C:\Users\Sanja\OneDrive\Desktop\Ai assistant coding> |

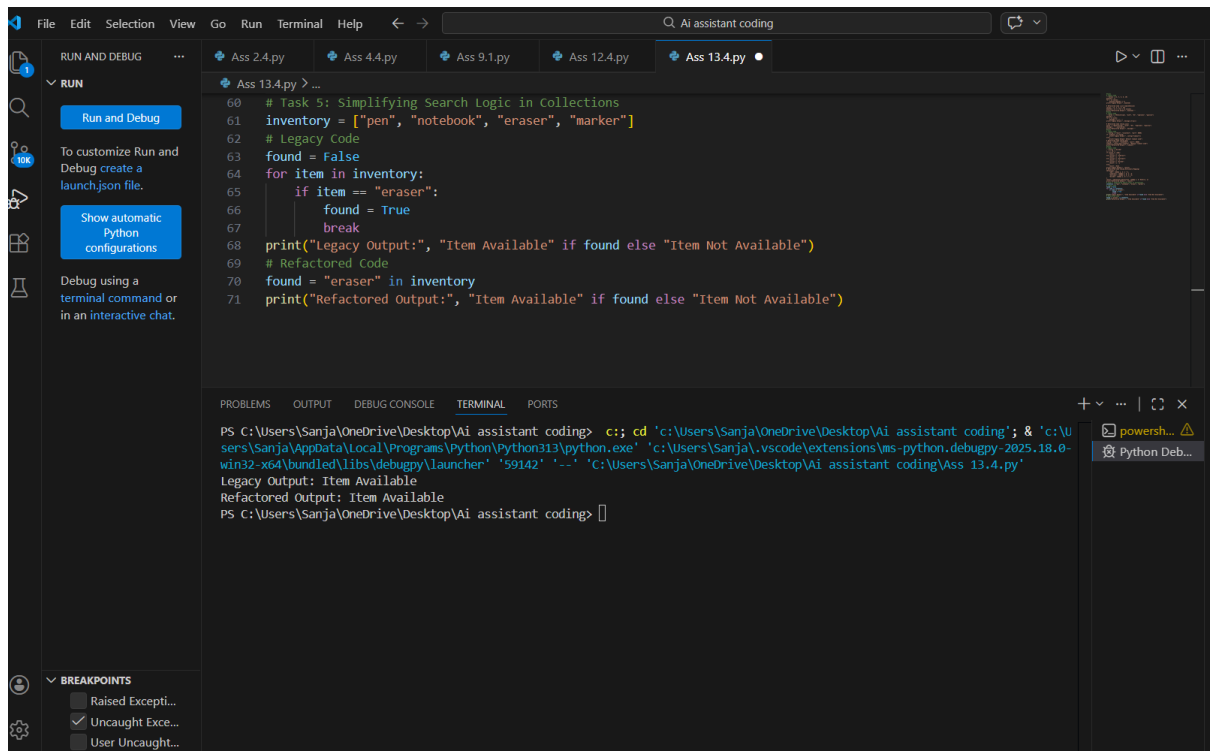
Observation:

The legacy code uses multiple if-elif conditions to perform operations. Refactoring with a dictionary mapping makes the code cleaner, easier to maintain, and more scalable.

Task5

Prompt:

Refactor the Python code that searches for an item in a list using a manual loop. Suggest a more concise and readable Python approach while keeping the output behavior the same.



Observation:

The legacy code searches for an item in a list using a manual loop. Refactoring with Python's built-in in operator simplifies the code and improves readability while keeping the same functionality.